
Per-Parameter Steps

Adagrad, RMSProp & Adadelta

Lesson 4 of the Optimizer Course

Giving every weight its own learning rate —
and finally killing the lopsided-bowl problem

Momentum helped the zig-zag, but one stubborn problem remained: a single learning rate for every weight, held hostage by the steepest direction. This lesson fixes that at the root — each weight gets its own step size, scaled automatically by its own history. We'll watch it flatten a 10-to-1 lopsided bowl into something that descends in a straight line.

1 The problem that momentum didn't fully solve

Momentum smoothed the bounce, but look back at every optimizer so far — GD, SGD, momentum — they all share *one* learning rate η for *all* weights. And we proved in Lesson 2 that this single rate is capped by the steepest direction: $\eta < 2/\lambda_{\max}$. So the calm, shallow directions are forced to crawl with the same tiny step that the wild, steep ones demand. That's the lopsided-bowl tax, and momentum only softened it.

What if, instead, **each weight had its own personal learning rate**? Big where the ground is calm, small where it's wild — automatically. Then no single direction could hold the others hostage. That's the idea behind the entire “Ada” family (Ada = *adaptive*), and it's the direct ancestor of Adam.

The big idea

Stop using one learning rate for everything. Give each weight a step size that *shrinks where its gradients are large and wild, and grows where they're small and calm*. Measure “large and wild” by keeping a running tally of each weight's past squared gradients.

2 Adagrad: divide by your history

Adagrad keeps, for each weight, a running *sum of its squared gradients* — call it G — and divides that weight's step by $\sqrt{G}$:

$$G \leftarrow G + g^2 \qquad w \leftarrow w - \frac{\eta}{\sqrt{G} + \varepsilon} g$$

Say it in English: “add this gradient's square to my running total G ; then take a step, but shrink it by how big my history has been.” Each weight has its own G , so each gets its own step size. The ε is a tiny number (10^{-8}) just to avoid dividing by zero.

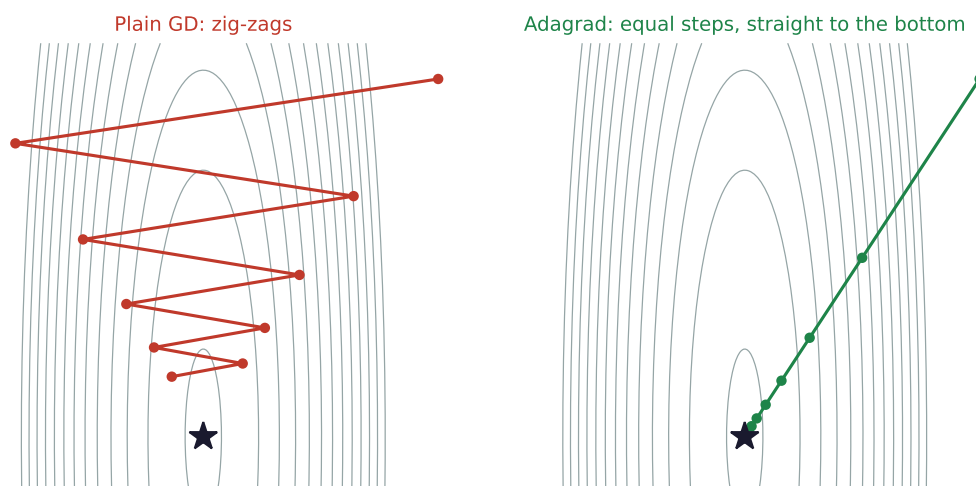
Here's the magic, on the same stretched bowl ($\nabla f = (10w_1, w_2)$, $\kappa = 10$) that tormented plain GD:

Let's actually do the numbers

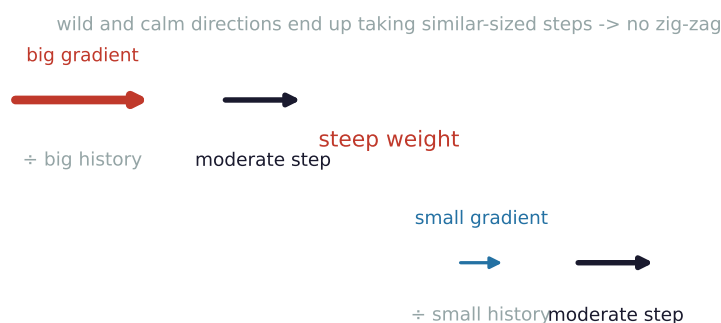
Start at $(1, 1)$, $\eta = 0.5$.

- **Step 1:** $g = (10, 1)$. Histories: $G = (100, 1)$. Steps: $\frac{0.5}{\sqrt{100}} \cdot 10 = 0.5$ and $\frac{0.5}{\sqrt{1}} \cdot 1 = 0.5$. **Both coordinates move by exactly 0.5** — even though w_1 's gradient was $10\times$ larger! Its huge step got divided by its huge history. New $w = (0.5, 0.5)$.
- **Step 2:** $g = (5, 0.5)$. $G = (125, 1.25)$. Steps: $\frac{0.5 \cdot 5}{\sqrt{125}} = 0.224$ and $\frac{0.5 \cdot 0.5}{\sqrt{1.25}} = 0.224$. **Equal again.** $w = (0.276, 0.276)$.
- **Step 3:** both move 0.120. $w = (0.156, 0.156)$.

The two coordinates descend in *lockstep*. The 10-to-1 lopsidedness has been **completely neutralised** — Adagrad turned a $\kappa = 10$ taco into something that descends like a round bowl.



Per-parameter scaling: divide each weight's step by its own gradient history



When you're actually training

This per-weight scaling was a breakthrough for **sparse data** — think early NLP, where most features (words) appear rarely. Rare features get tiny, infrequent gradients, so their histories stay small, so Adagrad gives them *big* steps when they finally fire. Common features get damped. It learns rare and common features at appropriate speeds automatically.

2.1 Adagrad's fatal flaw

Look again at that history G : it only ever *grows* (you keep adding squares, which are always positive). So $\sqrt{G}$ grows without bound, and the effective step size $\eta/\sqrt{G}$ marches relentlessly toward *zero*. Early in training that's fine. But on a long run, the steps shrink so much that learning grinds to a halt — often *before* the model has actually finished learning. Adagrad effectively has a built-in, unstoppable brake.

3 RMSProp: forget the distant past

The fix is almost embarrassingly simple. Adagrad's problem is that it remembers *all* history forever. RMSProp makes the memory *leaky*: instead of a growing sum, it keeps an **exponential moving average** of squared gradients — recent gradients count, old ones fade away.

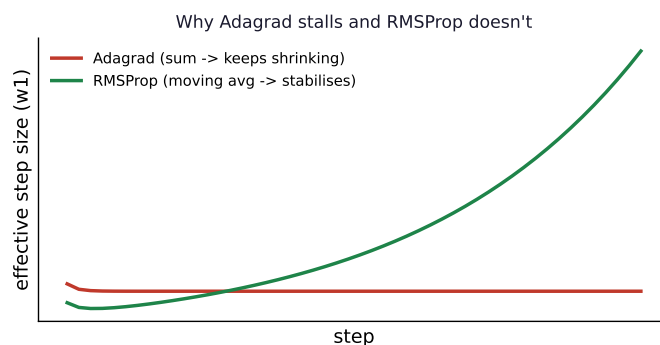
$$v \leftarrow \rho v + (1 - \rho) g^2 \qquad w \leftarrow w - \frac{\eta}{\sqrt{v} + \epsilon} g$$

Say it in English: same idea as Adagrad, but the history v is a *rolling average* (keep ρ , usually 0.9, of the old; add a little of the new) instead of an ever-growing sum. So it never blows up, and the step size never decays to zero.

Let's actually do the numbers

Same bowl, $\eta = 0.1$, $\rho = 0.9$, start $(1, 1)$.

- **Step 1:** $g = (10, 1)$, $v = 0.1 \cdot (100, 1) = (10, 0.1)$. Steps: $\frac{0.1 \cdot 10}{\sqrt{10}} = 0.316$ and $\frac{0.1 \cdot 1}{\sqrt{0.1}} = 0.316$. Equal again — $w = (0.684, 0.684)$.
- By step 6, $w = (0.144, 0.144)$ and still moving briskly. Crucially, run it 40 steps and it reaches the bottom — whereas Adagrad's steps would have shrivelled.



The plot says it all: Adagrad's effective step size keeps sliding toward zero (the unstoppable brake), while RMSProp's settles to a healthy, stable value and keeps making progress.

When you're actually training

RMSProp was the historical workhorse for **recurrent networks** (RNNs), where gradients vary wildly in scale and Adagrad's decay was especially deadly. You still see it, but in practice it's mostly been absorbed into **Adam** (next lesson), which is RMSProp *plus* momentum.

4 Adadelat: the no-learning-rate cousin

Worth knowing briefly. Adadelat also fixes Adagrad's decay with a rolling average, but adds a clever twist: it tracks a rolling average of the *past updates* too, and uses their ratio so that the units cancel out. The upshot is that Adadelat needs *no manually-set learning rate at all* — it derives the step scale from the problem's own history. It was an elegant idea and an ancestor of today's "learning-rate-free" methods (which we'll revisit in Lesson 9), but in practice RMSProp and Adam won out on simplicity and performance.

5 What you can do now, and what's next

You can now explain why one global learning rate is a problem, write the Adagrad and RMSProp updates, show by hand how per-parameter scaling equalises steps across steep and shallow directions, and explain Adagrad's decay-to-zero flaw and RMSProp's moving-average fix.

Next — Lesson 5: The Modern Default. We now have the two great ideas in hand: *momentum* (Lesson 3) gives a smart sense of *direction*; *RMSProp* (this lesson) gives a smart *per-weight step size*. What happens when you combine them — direction memory *and* per-weight scaling, with one elegant bias-correction fix on top? You get **Adam**, the optimizer that trains almost every model you've heard of. We'll build it from these exact pieces, by hand.